

⊕-Ядро 5D — Архитектура первого цифрового продукта (v0.1)

0. Манифест

"ГДЕЯ как 5-е измерение" объединяет 3D-пространство (X,Y,Z), время (T) и ось присутствия (G). Продукт — это интерфейс, который делает G операциональной: пользователь не только видит процессы, но **включает** их в смысловое поле.

Цель v0.1 (MVP, 6 недель): интерактивная панель (HUD) и движок сессий, где бизнес-команда управляет тремя метриками G-оси (I_Я, Q, τ) и проецирует их на 4D (ВЛАСТЬ, ОТНОШЕНИЯ, РЕЗУЛЬТАТ, РЕСУРС) и на 3D+T.

1. Персоны и сценарии (B2B)

- **Архитектор изменений** (CPO/Head of Transformation): настроить язык и ритм трансформации.
- **Лид-команды** (PM/Lead): проводит сессии резонанса, материализации, пустотного восстановления.
- **HR/Org Dev**: отслеживает культурную динамику, когерентность команд.

Ключевые сценарии: 1) Настройка профиля весов для задач (ВЛАСТЬ², близость, рывок, регенерация). 2) Проведение сессии: /align → /tune → /time → /lock → /void. 3) Карта состояния проекта в 5D: (X,Y,Z,T)×G и плитах (V,O,R,S).

2. Модель R₅ и вычисления G

$R_5 = (X,Y,Z,T,G)$, где $G = \varphi(I_{Я}, Q, \tau)$.

- **I_Я** — индекс центровки (0..1). Источники: самоотчёт + поведенческие маркеры.
- **Q** — качество отклика поля (0..1). Источники: обратная связь, NPS-аналоги, семантические маркеры.
- **τ** — темпоральная пластичность (0..1). Источники: фактический ритм, зазоры/буферы.

Эффективная мощность плиты: $P^* = P \cdot (w_i \cdot I_{Я} + w_q \cdot Q + w_\tau \cdot \tau)$.

Базовые профили весов: - ВЛАСТЬ²: (0.6, 0.25, 0.15) - Близость: (0.25, 0.6, 0.15) - Рывок: (0.3, 0.35, 0.35) - Регенерация: (0.25, 0.15, 0.6)

Энтропическая прозрачность: $E^e = (dS/dT) \times \tau \times Q \rightarrow 0$ — целевое окно.

3. Архитектура системы

Фронтенд: Next.js (React), TypeScript, Tailwind, shadcn/ui, Zustand (или Redux Toolkit) для состояния, Recharts для графиков.

Бэкенд: - FastAPI (Python) **или** Node.js (NestJS). Для v0.1 — FastAPI. - Postgres (реляционные данные), TimescaleDB-расширение для временных серий. - Vector DB (Qdrant/Weaviate) — хранение семантики обратной связи (для Q). - Message bus (Redis Streams) — события сессий.

Интеграции: - Импорт опросов/фидбека (CSV/Google Forms/Typeform), Jira/Linear (ритм T), Slack/Email (маркер Q), SSO (OIDC).

Слой: 1) **Core-движок метрик** (I, Я, Q, τ) и преобразование в P по плитам. 2) *Session Engine* — протоколы /align, /tune, /time, /lock, /void. 3) *Resonance NLP* — извлечение маркеров «факт-образ-тело» для Q. 4) *HUD 5D** — визуализация R_5 и плит.

4. Доменная схема (упрощённая)

```
classDiagram
    class Organization {id, name}
    class Team {id, name, org_id}
    class User {id, name, role, team_id}
    class Session {id, team_id, type, ts_start, ts_end}
    class Metric {id, session_id, IY, Q, tau, weights, plate_values}
    class Feedback {id, session_id, source, text, markers}
    class Project {id, team_id, name, stage}
    class Projection5D {id, project_id, X, Y, Z, T, G}

    Organization <|-- Team
    Team <|-- User
    Team <|-- Project
    Project <|-- Projection5D
    Team <|-- Session
    Session <|-- Metric
    Session <|-- Feedback
```

JSON-схема Metric (v0.1):

```
{
  "id": "uuid",
  "session_id": "uuid",
  "IY": 0.0,
  "Q": 0.0,
  "tau": 0.0,
  "weights": {"wi": 0.6, "wq": 0.25, "wt": 0.15},
  "plates": {"V": 0.0, "O": 0.0, "R": 0.0, "S": 0.0},
  "plates_eff": {"V": 0.0, "O": 0.0, "R": 0.0, "S": 0.0},
}
```

```
"entropy": {"dS_over_dT": 0.0, "Ee": 0.0},
"notes": ""
}
```

5. API (черновик)

POST /api/sessions — создать сессию

```
{"team_id": "uuid", "type": "align|tune|time|lock|void"}
```

POST /api/metrics — сохранить метрики

```
{"session_id": "uuid", "IY": 0.89, "Q": 0.86, "tau": 0.85, "weights": {"wi": 0.6, "wq": 0.25, "wt": 0.15}, "plates": {"V": 0.87, "O": 0.79, "R": 1.0, "S": 1.0}}
```

GET /api/projection5d?project_id= — текущая R₅-проекция

POST /api/feedback — маркеры для Q

```
{"session_id": "uuid", "source": "slack", "text": "..."} 
```

6. Алгоритмы (v0.1)

6.1. Индекс центровки I_Я - Шкала самооценки (Likert 1..7) → нормировка. - Поведенческие паттерны (фокус, границы, ясность роли) — чек-лист → boost/penalty.

6.2. Качество отклика Q - Семантика: извлечение триады «факт-образ-тело» из фидбека (легкий NLP). - Сходимость: коэффициент согласия стейкхолдеров. - Сигналы рынка: CTR/NPS/реплики — нормировка в 0..1.

6.3. Пластичность времени τ - Сопоставление планового/фактического ритма. - Доля буферов, глубина пауз, степень параллельности.

6.4. Плиты P - $P = P \cdot (w_i I_{Я}) + w_q Q + w_\tau \tau$. Базовые P берутся из текущих значений по плите.

6.5. Энтропическая прозрачность E^e - dS/dT — дисперсия контекстов/мнений по спринту. - $E_e = (dS/dT) \times \tau \times Q$ — целимся к 0.

7. HUD (UX-ядро)

Экраны: 1) **Matrix 5D** — сфера с центральной осью G, слайдеры I(Я), Q, τ; карта (V,O,R,S) и показатель P. 2) *Session Engine* — протоколы с кнопками-командами /align, /tune, /time, /lock, /void. 3) *Projection* — куб (X,Y,Z) с дорожками времени (T) и подсветкой уровня G. 4) *Resonance Feed** — текстовый поток фидбека и маркеров Q.

Статус-лампы: - I(Я): зелёный ≥ 0.85 ; жёлтый 0.7–0.85; красный < 0.7 . - Q: зелёный ≥ 0.85 ; жёлтый 0.7–0.85; красный < 0.7 . - τ: оптимум 0.8–0.9; иначе подсказки «растянуть/сжать».

8. Пример UI-команд (семантика)

- /align — Центровка: «Кто я здесь? Граница? Форма намерения?»
 - /tune — Резонанс: поле отражает три маркера (факт-образ-тело).
 - /time — Ритм: «спринт/дуга/пауза», установка τ.
 - /lock — Фиксация: критерий качества + дедлайн + подпись I(Я).
 - /void — Восстановление: «нулевое окно», снятие трения.
-

9. Безопасность и приватность

- RBAC: роли (Owner, Architect, Lead, Member, Viewer).
 - Шифрование: в покое (AES-256) и в транзите (TLS1.2+).
 - Анонимизация фидбека, настройка видимости маркеров Q.
-

10. MVP-объём (6 недель)

Недели 1–2: базовая модель метрик, Session Engine, простая Matrix 5D. **Недели 3–4:** HUD, Resonance Feed (легкий NLP), Projection. **Недели 5–6:** интеграция CSV/Typeform, экспорт отчёта, полировка UX.

Ключевые риски: перегрузка концепций → держим язык и UI минимальными.

11. KPI запуска

- ≥ 3 пилотные команды.
 - ≥ 2 успешных цикла внедрения (снижение $dS/dT \geq 30\%$).
 - $Q \geq 0.85$ на 2-й недели пилота.
-

12. Дорожная карта после MVP

- Семантические профили ролей (авто-подсказки по I(Я)/Q/τ).
- Игровые сценарии внедрения ("карты" для компаний).

- Партнёрские API (SDK для кастомных метрик).

13. Прототипные фрагменты (скелеты)

Компонент HUD (React/TS, семантика):

```
export default function GDEYA5DHUD() {
  return (
    <div className="p-6 grid gap-6">
      <header className="text-2xl font-semibold">⊕ HUD ⊖ Matrix 5D</header>
      <section className="grid md:grid-cols-2 gap-6">
        <div className="rounded-2xl p-4 shadow">
          <h2 className="text-lg">Ось G</h2>
          { /* Слайдеры IY, Q, tau */ }
        </div>
        <div className="rounded-2xl p-4 shadow">
          <h2 className="text-lg">Плиты (V,0, R, S)</h2>
          { /* Графики P* */ }
        </div>
      </section>
      <section className="rounded-2xl p-4 shadow">
        <h2 className="text-lg">Projection 5D</h2>
        { /* Куб XYZ, дорожка T, подсветка G */ }
      </section>
      <section className="rounded-2xl p-4 shadow">
        <h2 className="text-lg">Resonance Feed</h2>
        { /* Поток маркеров Q */ }
      </section>
    </div>
  );
}
```

Сервис расчёта метрик (псевдокод):

```
# inputs: P={V,0,R,S}, IY, Q, tau, weights=(wi,wq,wt)
# output: P_eff, Ee

coeff = wi*IY + wq*Q + wt*tau
P_eff = {k: P[k]*coeff for k in P}
Ee = (dS_over_dT)*tau*Q
return P_eff, Ee
```

14. Следующие шаги

1) Подтвердить словарь терминов UI. 2) Выбрать стек (FastAPI/TS), зафиксировать API-формы. 3) Подключить пилота (1 команда/проект). 4) Собрать прототип HUD и Session Engine. 5) Провести первую сессию: /align → /tune → /time → /lock → /void.